

```
<!DOCTYPE html> <html> <head> <meta charset="utf-8"> <title>Technical  
Presentation</title> <style> /* Marp theme styles will be injected here */ </style> </head>  
<body> <script type="text/markdown"> --- marp: true theme: alten paginate: true header:  
" " ---
```

Generative Newsletter: Technical Deep Dive

A comprehensive solution for automated newsletter generation.

System Architecture

A decoupled frontend and backend architecture for scalability and maintainability.

- **Frontend:** React (Vite + TypeScript)
- **Backend:** Python (Flask) on Google Cloud Run
- **Database:** Google Firestore & BigQuery
- **Async Processing:** Cloud Run Jobs & Pub/Sub
- **Storage:** Google Cloud Storage

Backend: Technology Stack

The backend is built with a robust and scalable stack.

- **Language:** Python 3
- **Framework:** Flask
- **Cloud Platform:** Google Cloud Platform (GCP)
- **Key GCP Services:**
 - Cloud Run (for services and jobs)
 - Firestore (for real-time data)
 - BigQuery (for analytics)
 - Pub/Sub (for event-driven tasks)
 - Vertex AI (for generative content)

Backend: Core Components

The backend is modular, with clear separation of concerns.

- `server/app.py` : Main Flask application, exposing all API endpoints.
- `newsletter_generation.py` : Handles the logic for creating daily newsletters, including AI-powered content generation.
- `digest_generation.py` : Manages the creation of weekly digests.
- `news_fetcher.py` : Responsible for fetching news from various sources.
- `news_search.py` : Powers the search functionality.

Backend: API Endpoints

A RESTful API provides access to the system's core functionalities.

- **GET /api/newsletters** : Retrieve newsletter history.
- **POST /api/newsletters/generate** : Trigger the generation of a new newsletter.
- **GET /api/digests** : Get the history of weekly digests.
- **POST /api/digests/generate** : Generate a new weekly digest.
- **GET /api/news/search?input=<query>** : Search for news articles.
- **PUT /api/newsletters/<id>** : Update an existing newsletter.

Frontend: Technology Stack

A modern, fast, and type-safe frontend.

- **Framework:** React
- **Build Tool:** Vite
- **Language:** TypeScript
- **Styling:** CSS / PostCSS / Tailwind CSS (based on config files)

Frontend: Core Components

The UI is built with a component-based architecture.

- `client/src/App.tsx` : The root component of the application.
- `client/src/pages` : Contains the main pages of the application:
 - `DailyNewsletter.tsx`
 - `WeeklyDigest.tsx`
 - `SearchResults.tsx`
- `client/src/components` : Reusable UI components like `NewsList.tsx` , `SearchBar.tsx` , and `Pagination.tsx` .
- `client/src/routes/AppRoutes.tsx` : Defines the application's routing.

Data Flow

From news source to the end user.

1. **News Fetching:** A scheduled Cloud Run job (`news_fetcher.py`) fetches news from various sources.
2. **Data Storage:** Articles are stored in Firestore.
3. **Newsletter Generation:** A user or a scheduled job triggers the newsletter generation process via the API.
4. **AI Processing:** Vertex AI is used to rank, cluster, and summarize news.
5. **Frontend Display:** The frontend fetches the generated newsletter from the API and renders it for the user.

Deployment & CI/CD

Automated and containerized deployment.

- **Containerization:** Both `client` and `server` have `Dockerfile`s for building container images.
- **Cloud Run:** Services are deployed to Cloud Run for serverless execution.
- **Deployment Scripts:** `ccloud_run_job/deploy_job.sh` automates the deployment process, enabling CI/CD pipelines.

Remaining Work: Web App

Focus on the weekly editing experience.

- Build a lightweight weekly editor to pick or upload the most impactful daily stories.
- Let curators add short notes that highlight why each story matters.

Remaining Work: Automation

Streamline enrichment, classification, and delivery.

- Auto-enrich top stories with customizable CI comments covering context and impact.
- Auto-classify news into sections such as general, competitors, financial reports, and research.
- Export HTML that fits directly into the existing newsletter tooling.

Questions?

```
</script> <script src="https://unpkg.com/marp-core/lib/marp.browser.js"></script>  
<script> const marp = new Marp(); const markdown =  
document.querySelector('script[type="text/markdown"]').textContent; const { html, css } =  
marp.render(markdown); document.head.querySelector('style').textContent = css;  
document.body.innerHTML = html; </script> </body> </html>
```